

> Lab - 1 | 22 January 2026

↳ 6 cells hidden

> Lab - 2 | 29 January 2026

↳ 14 cells hidden

> Lab - 3 | 05 February 2026

↳ 8 cells hidden

> Lab 4-12 Feb

↳ 2 cells hidden

✓ LAB 5

Name -Chirag Miglani Roll no. 1/23/SET/BCS/414 EXPERIMENT NO. 5

```
import numpy as np

# =====
# 1. XOR Dataset
# =====
X_in = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

Y_true = np.array([[0], [1], [1], [0]], dtype=float)

# =====
# 2. Activation Functions (ReLU + Sigmoid)
# =====
def relu(z):
    return np.maximum(0, z)

def relu_grad(z):
    return (z > 0).astype(float)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_grad(a):
    return a * (1 - a)

# =====
# 3. Network Structure
# =====
np.random.seed(2024)

inp = 2
h1 = 4
h2 = 3
out = 1

# He initialization for ReLU layers
W1 = np.random.randn(inp, h1) * np.sqrt(2 / inp)
b1 = np.zeros((1, h1))

W2 = np.random.randn(h1, h2) * np.sqrt(2 / h1)
```

```

b2 = np.zeros((1, h2))

# Xavier initialization for sigmoid output
W3 = np.random.randn(h2, out) * np.sqrt(1 / h2)
b3 = np.zeros((1, out))

lr = 0.08
epochs = 15000
clip_val = 1.0 # for gradient clipping

# =====
# 4. Training Loop
# =====
for epoch in range(epochs):

    # ----- Forward Pass -----
    Z1 = X_in @ W1 + b1
    A1 = relu(Z1)

    Z2 = A1 @ W2 + b2
    A2 = relu(Z2)

    Z3 = A2 @ W3 + b3
    A3 = sigmoid(Z3) # final prediction

    # ----- Binary Cross Entropy Loss -----
    loss = -np.mean(Y_true * np.log(A3 + 1e-9) + (1 - Y_true) * np.log(1 - A3 + 1e-9))

    # ----- Backpropagation -----
    dZ3 = A3 - Y_true
    dW3 = A2.T @ dZ3
    db3 = np.sum(dZ3, axis=0, keepdims=True)

    dA2 = dZ3 @ W3.T
    dZ2 = dA2 * relu_grad(Z2)
    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * relu_grad(Z1)
    dW1 = X_in.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    # ----- Gradient Clipping -----
    for grad in [dW1, dW2, dW3]:
        np.clip(grad, -clip_val, clip_val, out=grad)

    # ----- Parameter Updates -----
    W3 -= lr * dW3
    b3 -= lr * db3

    W2 -= lr * dW2
    b2 -= lr * db2

    W1 -= lr * dW1
    b1 -= lr * db1

    # ----- Logs -----
    if epoch % 2000 == 0:
        print(f"Epoch {epoch} | BCE Loss: {loss:.5f}")

# =====
# 5. Final Testing
# =====
print("\n===== FINAL XOR RESULTS =====\n")

for sample in X_in:
    s1 = relu(sample @ W1 + b1)
    s2 = relu(s1 @ W2 + b2)
    outp = sigmoid(s2 @ W3 + b3)

    print(f"Input {sample} → Output {outp[0][0]:.4f} → Class {int(outp[0][0] > 0.5)}")

```

```

Epoch 0 | BCE Loss: 0.74113
Epoch 2000 | BCE Loss: 0.00191
Epoch 4000 | BCE Loss: 0.00091
Epoch 6000 | BCE Loss: 0.00060

```

```
Epoch 8000 | BCE Loss: 0.00044  
Epoch 10000 | BCE Loss: 0.00035  
Epoch 12000 | BCE Loss: 0.00029  
Epoch 14000 | BCE Loss: 0.00025
```

```
===== FINAL XOR RESULTS =====
```

```
Input [0. 0.] → Output 0.0009 → Class 0  
Input [0. 1.] → Output 1.0000 → Class 1  
Input [1. 0.] → Output 1.0000 → Class 1  
Input [1. 1.] → Output 0.0000 → Class 0
```

Start coding or [generate](#) with AI.